Group Mini-Project Report

Cloud Computing Module — Mini-Project 2

Project Title	Cloud Service Log Analytics (From MapReduce to Ray)
Module	COMP3041J Cloud Computing
Submission Type	Group report
Anonymous Group Label	<Anonymous group label assigned for peer review>

Important anonymisation note: Do not include student names, student IDs, email addresses, repository usernames, cloud account details, or any other identifying information in this report. When referring to team members, use Member A, Member B, and Member C. Screenshots, logs, links, filenames, file paths, bucket names, and cloud-console outputs must be checked and anonymised before submission. Source code/repository information is submitted separately through Brightspace and should not be included in this peer-reviewed report if it reveals identity.

Abstract

This mini-project analysed a synthetic cloud service log dataset using Alibaba Cloud OSS, MapReduce, and Ray. The dataset contained 50,000 request records generated by several backend services, including authentication, payment, search, order, and notification services. The workflow followed the required pipeline: cloud object storage, MapReduce baseline analytics, Ray degraded-service detection, and comparison. The MapReduce jobs produced request count by service, server error count by service, and top 10 slow endpoints. The Ray extension detected degraded services based on slow request rate, server error rate, and Timeout error count. The project compared batch-style key-value aggregation with Ray-based parallel multi-metric analysis.

I. Project Overview and Workflow

This project analysed a synthetic cloud service log dataset from a simplified cloud-hosted application. Each log record included information such as timestamp, request ID, user ID, service name, endpoint, HTTP method, status code, response time, region, and error type. The services included authentication, payment, search, order, and notification services.

The group followed the required workflow: cloud service log dataset → cloud object storage → MapReduce baseline analytics → Ray extension analytics → comparison. First, the dataset was uploaded manually to Alibaba Cloud OSS as a private object to represent the cloud object-storage stage. For implementation and testing, the same dataset was kept locally as `dataset/cloud_logs.csv`.

Second, MapReduce was used to implement three baseline analytics tasks: request count by service, server error count by service, and top 10 slow endpoints. The Python mapper and reducer scripts were tested through local command-line streaming simulation and also used for Hadoop Streaming in a Docker-based Hadoop environment.

Third, Ray was used to implement degraded-service detection. The Ray script processed the dataset in parallel chunks and combined partial service-level statistics to determine whether each service was degraded.

Finally, the project compared MapReduce and Ray in terms of processing logic, output type, execution environment, and suitability for different analytics workloads.

II. Dataset and Cloud Object Storage

The project used the provided synthetic cloud service log dataset, which contained 50,000 request records. Each record included timestamp, request ID, user ID, service name, endpoint, HTTP method, status code, response time, region, and error type.

The dataset was uploaded manually to Alibaba Cloud OSS through the OSS console as a private object. The cloud object-storage step provided evidence that the dataset could be stored in a cloud-based storage service before analytics processing. For the implementation stage, the same dataset was stored locally as `dataset/cloud_logs.csv` and used consistently by the local MapReduce simulation, Hadoop Streaming execution, Ray degraded-service detection, and validation script.

Object storage is suitable for this workload because cloud service logs are commonly stored as large sequential files. Object storage provides scalable capacity, high durability, and low-cost storage without requiring complex database schema design. It also fits batch analytics workflows, where logs are stored first and later processed by tools such as MapReduce and Ray.

Selected evidence: include the anonymised Alibaba Cloud OSS upload screenshot, such as `oss_upload_evidence_anonymised.png` or `bucket_upload.png`. The screenshot hides bucket name, account information, usernames, and identifiable paths.

III. MapReduce Baseline Analytics

Three MapReduce jobs were implemented for the baseline analytics.

The first job counted the number of request records for each service. The mapper emitted `service_name 1` for each valid log record, and the reducer summed all values with the same service key.

```
bash-4.2$ cat /tmp/cloud_logs.csv | /tmp/request_count_mapper.py | sort | /tmp/request_count_reducer.py
auth-service    12121
notification-service    8412
order-service   10937
payment-service 7914
search-service  10616
bash-4.2$
```

The second job counted server errors by service. The mapper checked whether `status_code >= 500`. If the condition was true, it emitted `service_name 1`, and the reducer aggregated the counts for each service.

```
bash-4.2$ cat /tmp/cloud_logs.csv | python /tmp/server_error_mapper.py | sort | python /tmp/server_error_reducer.py
auth-service    436
notification-service    436
order-service   717
payment-service 1362
search-service  904
```

The third job identified the top 10 slow endpoints. A request was considered slow when `response_time_ms > 800`. The mapper emitted `service_name,endpoint 1` for each slow request, and the reducer summed the values and sorted the results to produce the top 10 endpoints.

```
bash-4.2$ cat /tmp/cloud_logs.csv | python /tmp/slow_endpoint_mapper.py | sort | python /tmp/slow_endpoint_reducer.py
search-service,/search/results  1174
search-service,/search/filter   1165
search-service,/search  1123
search-service,/search/autocomplete     1119
payment-service,/payments       577
payment-service,/payments/refund        526
payment-service,/payments/status        525
payment-service,/payments/confirm       506
order-service,/orders   256
order-service,/orders/history   229
```

Example MapReduce logic: for a record containing `payment-service` as the service name, the request-count mapper emitted `payment-service  1`. During the shuffle/sort stage, all values with the key `payment-service` were grouped together. The reducer then summed these values to produce the final request count for `payment-service`.

IV. Ray Extension Analytics

Ray was used to implement degraded-service detection. Unlike the MapReduce baseline, which produced separate count-based outputs, the Ray extension combined multiple service-level metrics to decide whether a service was degraded.

The dataset was split into chunks, and each Ray remote task processed one chunk of log records. For each service, the task calculated partial statistics, including total requests, slow requests, server errors, and Timeout errors. The driver program then merged all partial results into final service-level statistics.

A service was classified as degraded if it satisfied at least one of the following conditions: slow request rate greater than 20%, server error rate greater than 10%, or at least 5 Timeout errors. The final output used the format `service_name,reason`.

The detected degraded services included:

- `auth-service,repeated timeout errors`;
- `payment-service,high slow request rate; high server error rate; repeated timeout errors`;
- `search-service,high slow request rate; repeated timeout errors`.

```
================================================================================
Ray Degraded Service Detection
================================================================================
Dataset path: dataset\cloud_logs.csv
Output path: outputs\ray\degraded_services.txt
Total services: 5
Number of Ray chunks: 10
Runtime seconds: 7.900182

Service-level statistics:
auth-service: total=12121, slow=545, server_errors=436, timeouts=199, slow_rate=0.0450, server_error_rate=0.0360
notification-service: total=8412, slow=566, server_errors=436, timeouts=164, slow_rate=0.0673, server_error_rate=0.0518
order-service: total=10937, slow=1151, server_errors=717, timeouts=308, slow_rate=0.1052, server_error_rate=0.0656
payment-service: total=7914, slow=2134, server_errors=1362, timeouts=664, slow_rate=0.2696, server_error_rate=0.1721
search-service: total=10616, slow=4581, server_errors=904, timeouts=446, slow_rate=0.4315, server_error_rate=0.0852

Detected degraded services:
auth-service,repeated timeout errors
notification-service,repeated timeout errors
order-service,repeated timeout errors
payment-service,high slow request rate; high server error rate; repeated timeout errors
search-service,high slow request rate; repeated timeout errors
================================================================================
```

V. Validation of Results

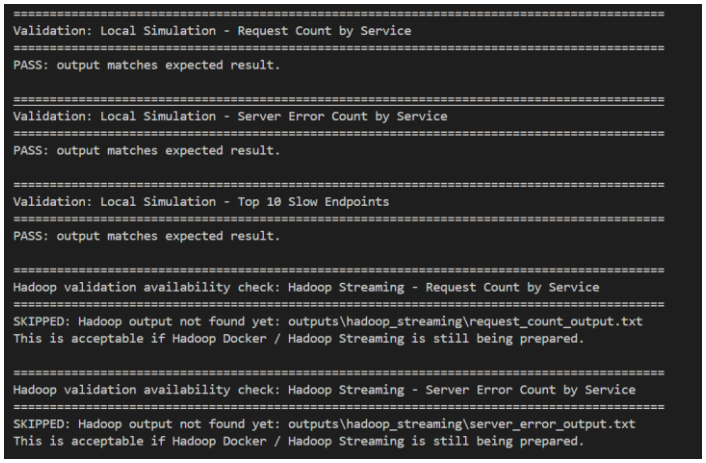
The group performed manual validation to check that the implementation outputs were correct.

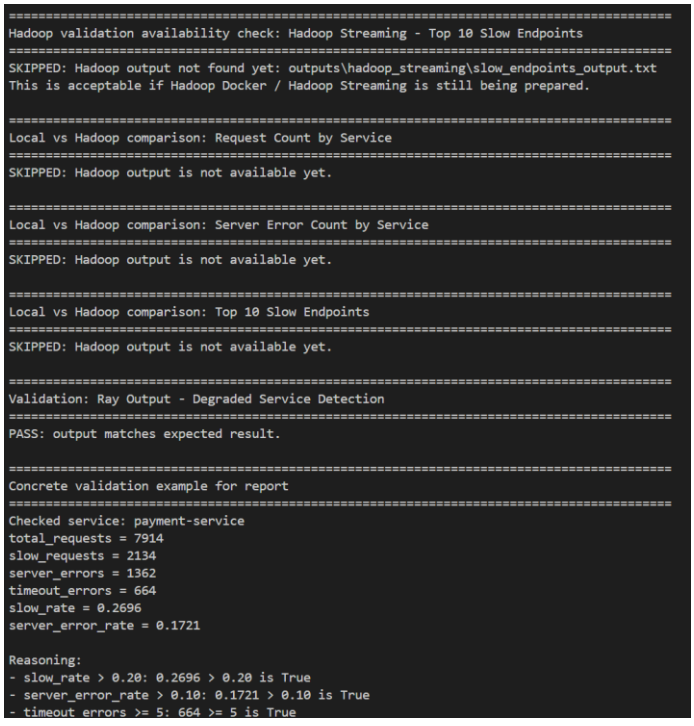
One validation example used the following input record:

The group validated the results using an independent pandas-based validation script. The script recalculated the expected request count by service, server error count by service, top 10 slow endpoints, and Ray degraded-service results directly from `dataset/cloud_logs.csv`.

For the MapReduce outputs, the validation script compared the local command-line streaming results with the pandas-calculated expected values. It was also designed to compare Hadoop Streaming outputs with the same expected values when those files were available. This allowed the group to check whether the local simulation and Hadoop Streaming results were consistent.

The Ray output was validated separately because it produced degraded-service reasons instead of simple count outputs. As one concrete validation example, `payment-service` had 7914 total requests, 2134 slow requests, 1362 server errors, and 664 Timeout errors. Its slow request rate was 0.2696, which is greater than 20%, and its server error rate was 0.1721, which is greater than 10%. It also had more than 5 Timeout errors. Therefore, the Ray output correctly classified it as degraded.





VI. Bounded Comparison and Analysis

Item	Required answer
MapReduce output evidence	The three MapReduce jobs produced the required baseline outputs. One representative row from each output is shown below: ● request count by service: `auth-service 12121`; ● server error count by service: `notification-service 436`; ● top 10 slow endpoints: `search-service,/search/results 1174`.
Ray output evidence	Ray output evidence: ● `auth-service,repeated timeout errors`; ● `payment-service,high slow request rate; high server error rate; repeated timeout errors`; ● `search-service,high slow request rate; repeated timeout errors`.
MapReduce logic	For the request-count job, an input record with payment-service as the service name was mapped to the intermediate key-value pair payment-service 1. During the shuffle/sort stage, all values with the same service key were grouped together. The reducer then summed all values for payment-service and produced the final count payment-service 7914.
Ray logic	The Ray implementation split the dataset into chunks and processed them using Ray remote tasks. Each task calculated partial statistics for each service, including total requests, slow requests, server errors, and Timeout errors. The driver program merged these partial results, calculated slow request rate and server error rate, then checked the degraded-service conditions to produce the final service_name,reason output.
Execution evidence	MapReduce execution environment: local command-line streaming simulation, with the same mapper and reducer scripts also tested through Hadoop Streaming in a Docker-based Hadoop environment. Ray execution environment: Ray local mode. Ray degraded-service detection processed 10 chunks and took 7.900182 seconds.
Conclusion	The implementation demonstrated that MapReduce is effective for structured batch aggregation tasks such as counting requests and errors. Ray provided greater flexibility for combining multiple metrics and performing parallel degraded-service analysis. The Ray implementation simplified the processing of multiple service conditions within one workflow. Overall, both approaches were useful, but Ray was more suitable for complex parallel analytics while MapReduce was better suited for traditional key-value aggregation tasks.

VII. Team Integration and Collaboration

The project was completed as an integrated group workflow rather than isolated tasks.

Member A prepared the Alibaba Cloud OSS storage step, worked on the Hadoop Docker / Hadoop Streaming execution environment, contributed to MapReduce code optimisation, and provided selected evidence screenshots for the MapReduce outputs and execution results.

Member B implemented and tested the local Python MapReduce mapper and reducer scripts, developed the Ray degraded-service detection script, created the pandas-based validation workflow, and helped organise the README and output structure.

The work was integrated through shared inputs and outputs. The dataset uploaded to Alibaba Cloud OSS was also kept locally as `dataset/cloud_logs.csv`, which was used by the MapReduce scripts, Ray script, and validation script. Member A's Hadoop Streaming work and MapReduce evidence were used to support the

MapReduce baseline section, while Member B's Ray and validation outputs were used to support the Ray extension and validation sections. The final comparison combined MapReduce output evidence, Ray degraded-service results, and validation evidence.

VIII. Group-Level Use of GenAI, If Applicable

Generative AI was used as a support tool during development and report preparation. It was used to clarify the project requirements, explain the relationship between local command-line streaming and Hadoop Streaming, support debugging, and improve the clarity of report wording.

One concrete example was the validation-stage encoding issue. Some PowerShell-generated output files were saved as UTF-16 LE and could not be read as UTF-8 by the validation script. GenAI was used to help identify the likely cause. The group then manually checked the error message, modified the validation script to support multiple encodings, and reran the validation.

GenAI suggestions were not accepted without testing. The final implementation, validation results, screenshots, and comparison were based on actual execution outputs produced by the group.